Ingegneria del Software

Michela Maria Tasca
Prof. Calvagna — UniCt

A.A. 2025/2026

Indice

1 Introduzione all'Ingegneria del Software **3**
1.1 Il Problema: Perché l'Ingegneria del Software? . . . 3
1.2 Definizione e Storia . . . 4
1.3 Il Processo di Sviluppo Software . . . 5
1.3.1 Modelli di Processo (Analisi dei Diagrammi) . . . 6
1.4 Forze Guida (Drivers) e Ruolo dell'Ingegnere . . . 7
1.4.1 Scienziato vs Ingegnere del Software . . . 7
1.5 La Qualità del Software . . . 8
1.6 Le Sfide Fondamentali in Ingegneria del Software . . . 8
1.6.1 1. I Requisiti Cambiano . . . 8
1.6.2 2. Il Software è intrinsecamente Complesso . . . 8
1.6.3 3. I Difetti sono Inevitabili . . . 9

2 Da C++ a Java **10**
2.1 Origini e Caratteristiche di Java . . . 10
2.2 Ambiente di Esecuzione: C++ vs Java . . . 10
2.3 Differenze Architetturali: C++ vs Java . . . 11
2.4 Identificatori e Puntatori . . . 13
2.5 Metodi e Override . . . 13
2.6 Esempio Pratico: Hello World e Diagramma UML . . . 13
2.6.1 Analisi del Class Diagram UML . . . 13
2.6.2 Implementazione Java del Modello . . . 14
2.7 Parole Chiave di Java . . . 15
2.7.1 Modificatori di Accesso . . . 15
2.8 Variabili e Tipi di Dato . . . 15
2.8.1 I 8 Tipi Primitivi . . . 15
2.8.2 Object Types . . . 16
2.9 Identità vs Uguaglianza: Il confronto tra Oggetti . . . 16
2.10 Metodi Statici e Input Utente . . . 16
2.10.1 Lettura Input con Scanner . . . 16
2.10.2 Gestione degli Errori: Exceptions . . . 17

3 Modellazione Object-Oriented con UML **18**
3.1 Introduzione . . . 18
3.1.1 Gestione della Complessità . . . 18
3.2 Sistemi, Modelli e Viste . . . 18
3.2.1 Tipologie di Modelli . . . 19

3.3 UML e le 4+1 Viste . . . 19
3.4 Elicitazione dei Requisiti . . . 20
3.5 Diagramma dei Casi d'Uso - Modellazione funzionale . . . 21
3.6 Il Progetto Software Orientato agli Oggetti . . . 22
3.6.1 Obiettivi Architetturali . . . 22
3.6.2 Tipologie di Diagrammi UML . . . 23
3.7 Diagramma delle Classi e degli Oggetti . . . 23
3.7.1 Attributi e Operazioni . . . 23
3.7.2 Oggetti . . . 23
3.8 Associazioni e Progettazione delle Relazioni . . . 24
3.8.1 Associazioni Qualificate . . . 25
3.8.2 Relazioni Speciali . . . 25
3.9 Esempio di Diagram Class . . . 27
3.10 Modellazione Comportamentale in UML . . . 27
3.11 Diagrammi di Stato (State Machines) . . . 28
3.11.1 Elementi Fondamentali . . . 28
3.11.2 Sintassi delle Transizioni e Tipi di Eventi . . . 29
3.11.3 Azioni Interne agli Stati . . . 30
3.11.4 Macchine a Stati Gerarchiche e Concorrenti . . . 30
3.12 Diagrammi di Attività . . . 31
3.12.1 Struttura e Corsie (Swimlanes) . . . 32
3.13 Diagrammi di Interazione . . . 33
3.13.1 Diagrammi di Collaborazione (Comunicazione) . . . 33
3.13.2 Diagrammi di Sequenza . . . 34

Capitolo 1

Introduzione all'Ingegneria del Software

1.1 Il Problema: Perché l'Ingegneria del Software?

Nello sviluppo del software, l'assenza di principi ingegneristici porta a fallimenti visibili e costosi. Un esempio classico è l'immagine di un **billboard del mago Lance Burton, interrotto da una finestra di dialogo di errore di Windows** .

Questo errore nel mondo reale evidenzia un difetto critico nella progettazione dei sistemi embedded o di produzione. In un ambiente di produzione, un sistema software non deve mai esporre dettagli di basso livello o bloccare l'esecuzione per attendere l'input di un utente inesistente. Questo fallimento sottolinea l'importanza della **robustezza**, della gestione corretta delle eccezioni e della configurazione adeguata dell'ambiente di sviluppo, concetti cardine dell'Ingegneria del Software per garantire la *dependability* del sistema.

Altri fallimenti storici dovuti a bug software includono:

- **Mars Climate Orbiter (1999):** Perso a causa di un errore di riuso dei componenti (mancata conversione tra sistema metrico decimale e imperiale).
- **Mars Polar Lander (1999):** Fallito per una violazione delle precondizioni nel software.

- **USS Yorktown (1997):** L'incrociatore è rimasto bloccato in mare per ore a causa di un'eccezione non gestita (divisione per zero) inserita da un operatore, che ha causato un buffer overflow.
- **Space Shuttle Columbia:** Il primo lancio fu ritardato per un problema di sincronizzazione causato dalla modifica di un fattore di ritardo da 50 a 80 millisecondi anni prima.

1.2 Definizione e Storia

Il termine *Software Engineering* è nato nel 1968 durante la Conferenza NATO di Garmisch, per rispondere alla "crisi del software": i progetti non rispettavano le scadenze, superavano i budget e producevano software di bassa qualità.

Una figura chiave è stata **Margaret Hamilton**, ritratta accanto a una **pila di codici stampati su carta alta quanto lei**.

La foto mostra il codice sorgente del software di volo delle missioni Apollo (Apollo Guidance Computer). La Hamilton ha coniato il termine "Ingegneria del Software" lottando affinché lo sviluppo del codice ricevesse lo stesso rispetto, legittimità e rigore metodologico riservato all'hardware, permettendo all'umanità di arrivare sulla Luna senza crash di sistema critici.

Definizione 1.1: Ingegneria del Software (Definizione Preliminare)

La disciplina che studia il processo di costruzione di un prodotto software.

Per comprendere la portata di questa disciplina, bisogna considerare la dimensione tipica dei prodotti software, il numero di persone necessarie per costruirli, e l'impatto critico di eventuali ritardi o bug (che possono costare vite umane o miliardi di dollari, come nel caso del **Space Shuttle Software**, che superò il budget di 10 miliardi di dollari ed ebbe 3 anni di ritardo per un errore di sincronizzazione).

Definizione 1.2: Ingegneria del Software (Definizione Completa)

L'Ingegneria del Software è un insieme di tecniche, metodologie e strumenti che aiutano nella produzione di un sistema software di **alta qualità**, rispettando un **budget** e una **scadenza** prefissati, gestendo al contempo il **cambiamento** continuo dei requisiti.

1.3 Il Processo di Sviluppo Software

Definizione 1.3: Processo Software

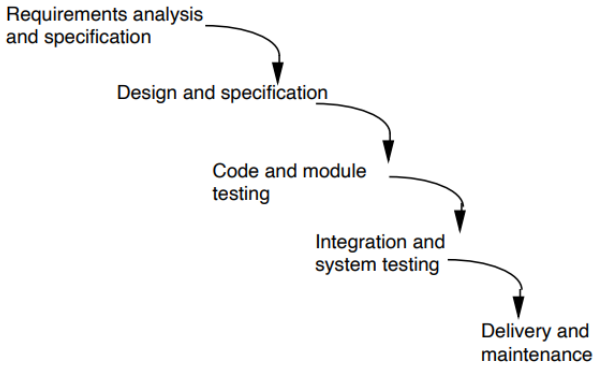
Un insieme di attività e le relative relazioni che supportano lo sviluppo o l'evoluzione di un sistema software. Risponde a domande su quali attività selezionare, le loro dipendenze e la loro pianificazione.

Le attività generiche in tutti i processi software sono:

- **Analisi e Specifica:** Definire cosa deve fare il sistema e i suoi vincoli.
- **Sviluppo (Development):** La produzione vera e propria del codice.
- **Validazione:** Verificare che il software corrisponda a ciò che il cliente desidera.
- **Manutenzione / Evoluzione:** Modificare il software in risposta a nuove richieste.

1.3.1 Modelli di Processo (Analisi dei Diagrammi)

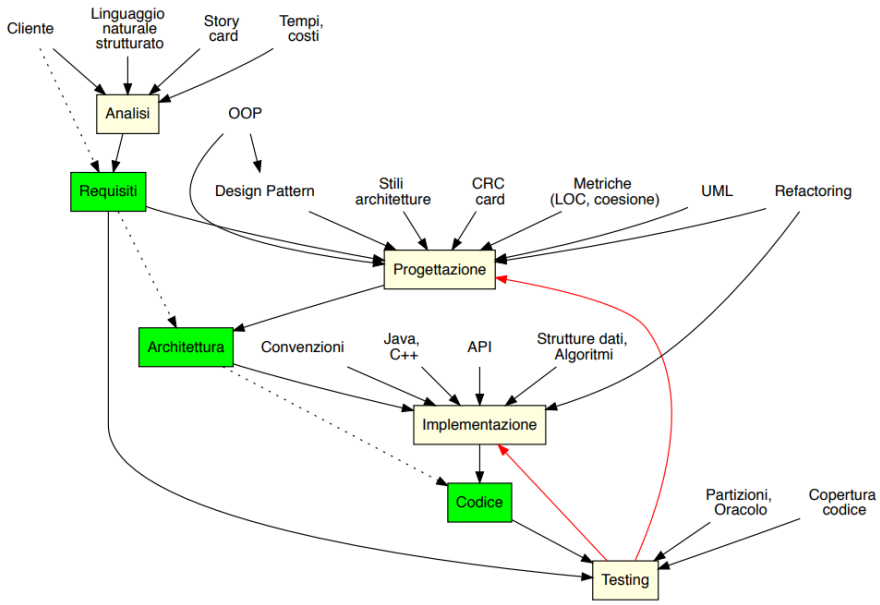
Il Modello a Cascata (Waterfall Model)



Il diagramma del ciclo di vita classico mostra un flusso sequenziale in cui ogni fase produce gli artefatti necessari per la successiva:

1. **Requirements analysis and specification** (Analisi dei requisiti).
2. **Design and specification** (Progettazione).
3. **Code and module testing** (Implementazione e test di unità).
4. **Integration and system testing** (Integrazione e collaudo).
5. **Delivery and maintenance** (Rilascio e manutenzione).

Il Processo di Sviluppo (Enhanced Model)

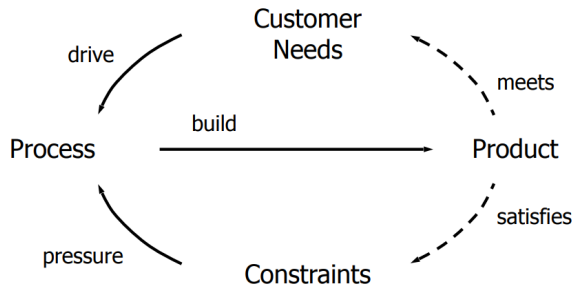


Il diagramma del modello esteso illustra un approccio più iterativo e basato su concetti moderni:

- **Attori e Input:** Il *Cliente* fornisce requisiti sotto forma di *Linguaggio naturale strutturato*, *Story card* o *Use case*.
- **Fase di Analisi:** Produce i documenti di specifica.
- **Fase di Progettazione:** Riceve input dall'analisi e applica concetti come *OOP, Design Pattern, Architetture software e CRC card*.
- **Fase di Implementazione:** Traduce il design in codice utilizzando *Convenzioni, API, Strutture dati e Algoritmi*.
- **Fase di Testing:** Utilizza artefatti come *Partizioni, Oracoli e Copertura del codice* per validare il prodotto, con un ciclo di feedback (*Refactoring* e *UML*) che torna verso la progettazione e l'implementazione.

1.4 Forze Guida (Drivers) e Ruolo dell'Ingegnere

Il processo e il prodotto sono influenzati da diverse forze (Drivers) interconnesse:



- **Customer Needs (Requisiti dei clienti/utenti):** Guidano il processo.
- **Process (Team e Processi):** Costruiscono il prodotto.
- **Product (Tecnologia):** Soddisfa i vincoli e le necessità.
- **Constraints (Contesto, Leggi, Budget):** Esercitano pressione sul processo.

Un cambiamento significativo in una di queste forze (es. passare da 10 rilasci al giorno a 1 rilascio ogni 100 giorni per motivi normativi) altera l'intera architettura e struttura del team. Come suggerito da **Jeff Dean** (Google), una variazione di **10x** nelle dimensioni di un progetto rende le vecchie soluzioni sub-ottimali, mentre una variazione di **100x** richiede approcci e strutture organizzative radicalmente nuove.

1.4.1 Scienziato vs Ingegnere del Software

- **Computer Scientist:** Dimostra teoremi, definisce linguaggi, lavora senza vincoli di tempo stringenti.
- **Software Engineer:** Lavora in domini applicativi multipli, deve risolvere i problemi del cliente in tempi ridotti (es. 3 mesi), integrando strumenti e metodologie mentre i requisiti e le tecnologie cambiano. Richiede forti doti comunicative, di astrazione, di modellazione e capacità di lavorare in team.

1.5 La Qualità del Software

La valutazione del software si basa su criteri operativi:

- **Correttezza:** Il software fa ciò che il cliente vuole? (*Validazione funzionale*). Soddisfa le specifiche tecniche? (*Verifica tecnica*).
- **Efficienza, Dependability (Sicurezza/Affidabilità), Usabilità.**
- **Manutenibilità (Maintainability):** È il criterio più critico e costoso. Poiché il software non ha parti fisiche, è intrinsecamente modificabile. Se un software ha successo, sopravviverà all'hardware originale e dovrà inevitabilmente evolvere per adattarsi a nuovi requisiti.

1.6 Le Sfide Fondamentali in Ingegneria del Software

Le tre sfide principali che gli ingegneri devono affrontare sono: **Cambiamento**, **Complessità** e **Difetti**.

1.6.1 1. I Requisiti Cambiano

I requisiti degli utenti (bisogni, obiettivi) mutano nel tempo. Il fallimento di un prodotto nel soddisfare l'utente, pur avendo seguito le specifiche iniziali, impone di adottare un **Processo** flessibile in grado di assorbire i cambiamenti.

1.6.2 2. Il Software è intrinsecamente Complesso

Il software è complesso perché è difficile da comprendere e modificare.

Definizione 1.4: Complessità Accidentale vs Complessità Intrinseca

Complessità Accidentale: È legata a scelte di implementazione inadeguate. Può essere rimossa pulendo il codice (refactoring), cambiando linguaggio o ripagando il *Debito Tecnico*.
Complessità Intrinseca: Rimane anche dopo aver rimosso l'accidentale. Deriva dal fatto che il comportamento a *run-time* (dinamico) è invisibile rispetto al testo statico del codice sorgente.

L'imprevedibilità del comportamento dinamico rende impossibile anticipare tutti gli stati del programma. Per gestire la complessità intrinseca, è necessario applicare **Design Pattern**, progettare sistemi modulari, mantenere un **basso accoppiamento** (low coupling) e un'**alta coesione** (high cohesion).

Esempio di Complessità Intrinseca: Dato il seguente frammento di codice la domanda è: il loop finisce?

```
do j++;
    while a[j] >= pivot;
```

La risposta dipende dal valore di `j`, di `pivot`, e dell'elemento dell'array. La conclusione è che lo stesso codice potrebbe, potenzialmente avere infiniti outcome.

1.6.3 3. I Difetti sono Inevitabili

Come dimostrato dagli esperimenti di **George Miller** (anni '50), la mente umana può gestire simultaneamente circa 7 blocchi di informazioni (*chunks*). Oltre questa soglia, insorgono confusione ed errori. Poiché il software supera facilmente questa soglia, per garantire la qualità bisogna **Testare (Validare)** o **Dimostrare (Verificare)** la correttezza del prodotto.

Definizione 1.5: Fault (Difetto) vs Failure (Fallimento)

Fault (Difetto o Bug): È un'anomalia statica presente nel codice sorgente o nei documenti di specifica.
Failure (Fallimento): È il comportamento anomalo scatenato a *run-time* (dinamicamente) da un Fault. Ad esempio, un'app che va in *crash* subisce un failure; il failure si manifesta solo in determinate condizioni, ma il fault è sempre presente nel codice.

Capitolo 2

Da C++ a Java

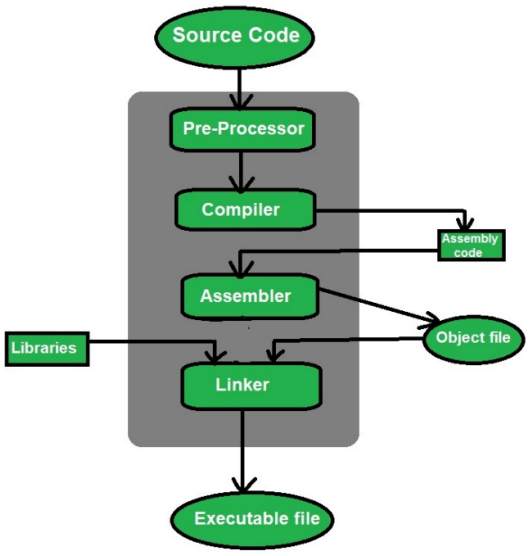
2.1 Origini e Caratteristiche di Java

Java è stato sviluppato nei primi anni '90 da un team della Sun Microsystems guidato da James Gosling.

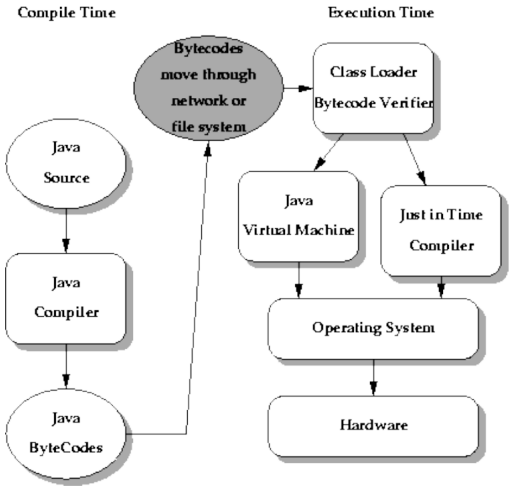
- Originariamente chiamato **Oak** (ispirato a un albero fuori dall'ufficio di Gosling).
- Inizialmente pensato per la programmazione di smart TV e altri dispositivi, è stato rapidamente riadattato per il Web emergente (prima release nel 1995).
- È uno dei linguaggi più popolari al mondo, ampiamente usato nello sviluppo di software enterprise.
- Possiede una vasta libreria di funzionalità integrate, garantisce buone prestazioni ed elevata portabilità.
- È il linguaggio predefinito per i dispositivi mobili Android e per giochi come Minecraft.
- Appartiene alla famiglia dei linguaggi **Object-Oriented (OO)**: i programmi sono organizzati in unità chiamate classi e oggetti.

2.2 Ambiente di Esecuzione: C++ vs Java

Di seguito due schemi che mettono a confronto i processi di compilazione ed esecuzione tra C++ e Java. Questo confronto è cruciale per comprendere le differenze di portabilità e performance.



(a) C/C++ Program Environment



(b) Java Program Environment

Il primo diagramma, che si riferisce all'esecuzione del codice in C/C++ mostra un processo di compilazione nativa diretta:

1. **Source Code:** Il codice sorgente passa al **Pre-Processor**.
2. **Compiler:** Il codice pre-processato viene tradotto in assembly.
3. **Assembler:** Genera l'**Object File**.
4. **Linker:** Collega le librerie esterne e genera l'**Executable File**.
5. L'eseguibile viene poi eseguito direttamente sull'hardware. Questo lo rende veloce ma dipendente dalla piattaforma (non portabile).

Il diagramma per Java mostra invece un approccio ibrido (compilazione + interpretazione):

1. **Compile Time:** Il **Java Source** viene processato dal **Java Compiler**, che genera i **Bytecodes**. Questi bytecodes possono viaggiare attraverso la rete o il file system.
2. **Execution Time:** Il **Class Loader** e il **Bytecode Verifier** caricano il bytecode nella **Java Virtual Machine (JVM)**.
3. La JVM utilizza un **Just in Time (JIT) Compiler** per tradurre il bytecode in istruzioni macchina specifiche per l'**Operating System** e l'**Hardware** sottostante. Questo garantisce la portabilità ("Write Once, Run Anywhere").

2.3 Differenze Architetturali: C++ vs Java

Il confronto tra C++ e Java evidenzia filosofie di design profondamente diverse, specialmente nella gestione della memoria e nell'Object Orientation.

- **Paradigma OOP Puro:** In Java tutto deve risiedere all'interno di una classe (o interfaccia). **Non esistono variabili globali** libere. Una variabile `public static` simula il concetto di variabile globale, ma resta confinata in un namespace di classe.

- **Tipizzazione Fortemente Controllata (Strong Type Checking):** Come il C++, i tipi devono essere compatibili a tempo di compilazione. Java aggiunge un rigoroso controllo sul **tipo dinamico** a runtime: un identificatore (reference) può puntare a oggetti di classi derivate, ma l'effettivo uso deve rispettare le gerarchie di ereditarietà.
- **Ereditarietà:** Java **non supporta l'ereditarietà multipla** tra classi per evitare il "Problema del diamante" (ambiguità nella risoluzione dei metodi). Questo design limitante è aggirato dal supporto all'implementazione di interfacce multiple.
- **Radice della Gerarchia:** Tutte le classi Java derivano implicitamente dalla superclasse `Object`.
- **Puntatori e Identificatori:** In Java gli identificatori a tipi non primitivi **sono di fatto puntatori** (reference), ma non è consentita l'aritmetica dei puntatori (es. `b++` è illegale). L'operatore `.` sostituisce l'uso di `->` del C++.
- **Gestione della Memoria:** A differenza del C++ dove la memoria allocata dinamicamente con `new` richiede una `delete` esplicita, in Java la deallocazione è delegata al **Garbage Collector**, che interviene automaticamente per eliminare gli oggetti non più referenziati, prevenendo i memory leak.
- Il significato del separatore punto (`.`) in Java è lo stesso dell'operatore freccia (`->`) in C++.
- **Gli identificatori in Java sono tutti puntatori** (riferimenti agli oggetti).
- Non ci sono entità esterne agli oggetti: nulla può essere definito al di fuori di un costrutto di classe o interfaccia. Di conseguenza, **non esistono variabili globali** (anche se una classe con un attributo statico funge da pseudo-globale).
- Non serve il punto e virgola (`;`) alla fine dei blocchi dichiarativi delle classi `{}`.
- In una classe vengono inserite contestualmente sia l'astrazione (l'Abstract Data Type, ADT) che la sua implementazione.
- Java è **fortemente tipizzato (strong type checking)** come il C++. I tipi ai lati di un assegnamento devono essere formalmente compatibili a tempo di compilazione.
- Java esegue anche un controllo sul **tipo dinamico**: un identificatore può puntare a oggetti di tipo diverso in momenti diversi dell'esecuzione, ma il tipo effettivo a runtime deve sempre essere compatibile con il tipo a cui lo si assegna.
- Java **non supporta l'ereditarietà multipla** per evitare il "Problema del diamante". Questo comportamento è però replicabile tramite l'uso delle **interfacce**.
- La radice della gerarchia delle classi in Java è unica ed è la classe **Object**.
- La libreria standard si trova nel dominio `java.*`.
- È consentita una sola classe pubblica per file (salvo classi nidificate) e il file deve avere lo stesso nome della classe.
- Il metodo `main` deve obbligatoriamente essere definito all'interno di una classe.
- L'istruzione `import` (a differenza degli `#include` in C++ che importano interi file header) indica gruppi di librerie o singole classi per nome composto.

2.4 Identificatori e Puntatori

- **In C++:** L'istruzione `T a;` alloca direttamente una struct/oggetto `T`. L'istruzione `T *b = new T();` alloca dinamicamente un puntatore e richiede l'uso esplicito di `delete(b)` per deallocare la memoria. È consentita l'aritmetica dei puntatori (es. `b++`). L'accesso ai membri avviene tramite `a.m()` o `b->m()`.
- **In Java:** L'istruzione `T a;` dichiara solo un identificatore (riferimento), ma non alloca l'oggetto. L'istruzione `T a = new T();` alloca dinamicamente l'istanza. L'aritmetica dei puntatori è **illegale**. L'assegnamento tra oggetti (`a = b`) copia il riferimento, non l'oggetto in sé. La deallocazione della memoria è gestita automaticamente dal **Garbage Collector**. L'accesso ai membri avviene **sempre e solo** col punto (`a.m()`).

2.5 Metodi e Override

La gestione del dispatching dei metodi (risoluzione delle chiamate a metodo) varia tra i due linguaggi:

- **Java:** Il dispatch è a **run-time** ed è **automatico**. Non è necessaria alcuna keyword per l'override. I metodi non virtuali (che non possono subire override) sono quelli contrassegnati come `final`, `static` o `private`.
- **C++:** Il dispatch avviene a run-time **solo se** è presente la keyword `virtual`. Di default, i metodi sono non virtuali e non richiedono keyword specifiche.

2.6 Esempio Pratico: Hello World e Diagramma UML

Nelle slide viene analizzato il classico esempio "Hello World" con l'ausilio di un Class Diagram UML, fondamentale in Ingegneria del Software per modellare la struttura statica del sistema.

2.6.1 Analisi del Class Diagram UML

Nella progettazione software, i Class Diagram UML modellano la **struttura statica** di un sistema, definendo le entità, i loro attributi (stato) e metodi (comportamento). Di seguito il diagramma UML della classe `HelloWorld`:

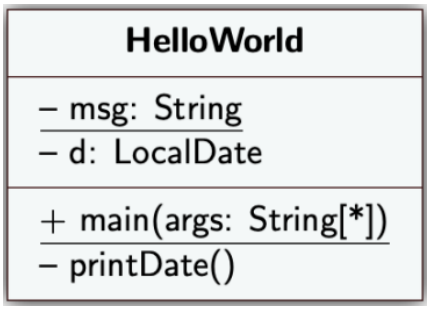


Figura 2.2: "Hello World" UML Diagram

Il diagramma UML rappresenta la classe `HelloWorld` con la seguente struttura formale:

- **Nome Classe:** `HelloWorld`
- **Attributi (Fields):**
  - `- msg: String`: La visibilità privata (`-`) impone l'incapsulamento. Il campo è di tipo String.
  - `- d: LocalDate`: Variabile privata per conservare un'istanza temporale.
- **Metodi (Operations):**
  - `+ main(args: String[])`: Metodo pubblico (`+`) che funge da entry point dell'applicazione. Riceve un array di stringhe. Sottolineato nel diagramma (spesso indica un membro `static`).
  - `- printDate()`: Metodo privato senza parametri, deputato a un'azione interna di supporto.

2.6.2 Implementazione Java del Modello

Di seguito la traduzione in codice Java del modello sopra UML descritto:

```java
import java.time.LocalDate; // Indica dove trovare la classe LocalDate

public class HelloWorld {
    // Dichiarazione campi (stato)
    private static final String msg = "Lezione di Ingegneria del Software";
    private LocalDate d;

    // Entry point del programma
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println(msg);

        final HelloWorld world = new HelloWorld(); // Creazione oggetto
        world.printDate(); // Chiamata al metodo d'istanza
    }

    // Metodo privato d'istanza
    private void printDate() {
        d = LocalDate.now(); // Chiamata a metodo statico
        System.out.println(d);
    }
}
```

L'output sarà il seguente:

```
Output
Hello World
```

```
Lezione di Ingegneria del Software
2026-03-05
```

2.7 Parole Chiave di Java

- `class`: Permette di definire un tipo di dato astratto e le sue relative istanze.
- `final`: Definisce un campo o una variabile che non può essere riassegnata (costante). Applicata a una classe, impedisce l'ereditarietà. Applicata a un metodo, impedisce l'override.
- `import`: Indica al compilatore dove trovare la definizione di una classe esterna.
- `new`: Alloca dinamicamente la memoria e crea un'istanza di una classe.
- `private` e `public`: Modificatori di accesso per incapsulamento (vedi sezione successiva).
- `static`: Dichiara che un campo o un metodo appartiene alla *classe* nel suo complesso, e non alla singola istanza.
- `void`: Indica che un metodo non restituisce alcun valore.

2.7.1 Modificatori di Accesso

I modificatori determinano la visibilità di classi, metodi e attributi:

- **Public:** Visibile a Class, Package, Subclass, Global.
- **Protected:** Visibile a Class, Package, Subclass. Non globale.
- **Default (Package-private):** Visibile a Class, Package.
- **Private:** Visibile esclusivamente all'interno della Class.

2.8 Variabili e Tipi di Dato

In Java, ogni variabile ha un tipo dichiarato e può contenere solo valori appartenenti a quel tipo.

2.8.1 I 8 Tipi Primitivi

1. `int`: Intero con segno a 32-bit (da -2^{31} a $2^{31}-1$).
2. `short`: Intero con segno a 16-bit (da -2^{15} a $2^{15}-1$). *Nota: nelle slide c'è un refuso (2^{16}), il range corretto è basato su 15 bit per il valore e 1 per il segno.*
3. `long`: Intero con segno a 64-bit.
4. `byte`: Intero con segno a 8-bit (da -128 a 127).
5. `double`: Virgola mobile a 64-bit ($\pm 1.7E308$).
6. `float`: Virgola mobile a 32-bit ($\pm 3.4E38$).

7. `char`: Carattere testuale singolo, Unicode a 16-bit.
8. `boolean`: Valore logico (`true` o `false`).

2.8.2 Object Types

Qualsiasi tipo che non è primitivo è un Object Type (es. `String`, per sequenze di caratteri). Per convenzione, iniziano con la lettera maiuscola. La variabile che contiene un Object Type è in realtà un **riferimento** (puntatore) all'oggetto sottostante in memoria.

2.9 Identità vs Uguaglianza: Il confronto tra Oggetti

Un principio base nella programmazione Java è la differenza tra uguaglianza dei riferimenti e uguaglianza dei valori:

- **Operatore == (Reference Equality):** Valuta se due variabili puntano esattamente **alla stessa istanza in memoria**.
- **Metodo `.equals()` (Value Equality):** Valuta se due oggetti contengono **dati semanticamente equivalenti**, indipendentemente dalle loro locazioni in memoria. Deve essere sovrascritto (override) dallo sviluppatore nella classe specifica.

```
String s1 = new String("Tyrannosaurus Rex");
String s2 = new String("Tyrannosaurus Rex");

if (s1 == s2) // FALSE: sono due oggetti distinti in memoria
    System.out.println("Stesso oggetto");

if (s1.equals(s2)) // TRUE: i caratteri che li compongono sono identici
    System.out.println("Valore identico");
```

2.10 Metodi Statici e Input Utente

I metodi statici sono blocchi di codice nominati, unici a livello globale per la classe e immediatamente eseguibili senza istanziare l'oggetto.

2.10.1 Lettura Input con Scanner

L'interazione con l'utente a riga di comando si gestisce tramite la classe `Scanner` (appartenente al package `java.util`). Si inizializza passandogli un flusso di input, solitamente `System.in` (standard input).

```
import java.util.Scanner;

public class ScannerDemo {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in); // Istanziazione dello Scanner
        System.out.println("Come ti chiami?");
```

```
7         String name = input.nextLine(); // Lettura di una riga di
    testo
8         System.out.printf("Ciao, %s.", name);
9     }
10 }
```

2.10.2 Gestione degli Errori: Exceptions

L'ingegneria del software richiede sistemi robusti e fault-tolerant. Se un utente inserisce un dato incompatibile (es. digita "dieci" quando è atteso un `int` tramite `nextInt()`), l'esecuzione non causa un crash di sistema irreversibile o un core dump (come avviene spesso in C), ma la JVM solleva una **Eccezione** (in questo caso, `InputMismatchException`). Le eccezioni sono oggetti che rappresentano un'anomalia a run-time e che possono essere "catturate" e gestite (*caught and handled*) all'interno del codice per permettere al programma di riprendersi dall'errore gracefully.

Capitolo 3

Modellazione Object-Oriented con UML

3.1 Introduzione

La modellazione del software è diventata una necessità imprescindibile a causa della crescente complessità dei sistemi moderni. Ad esempio, sistemi operativi come Windows superano i 40 milioni di righe di codice, una quantità impossibile da gestire integralmente per un singolo programmatore. Il codice sorgente, inoltre, non è uno strumento di comunicazione efficace per chi non lo ha scritto.

3.1.1 Gestione della Complessità

Il **Object-Oriented Design (OOD)** fornisce tre strumenti fondamentali per gestire la complessità:

1. **Astrazione**: Capacità di ignorare i dettagli non essenziali per concentrarsi sugli aspetti rilevanti. Questo risponde al limite umano di gestire solo 7 ± 2 concetti simultaneamente (fenomeno del chunking).
2. **Decomposizione**: Suddivisione di un problema complesso in parti più piccole e gestibili.
3. **Gerarchia**: Organizzazione delle astrazioni in strutture ordinate.

3.2 Sistemi, Modelli e Viste

Definizione 3.1: Modello

Un **modello** è un'astrazione che descrive un sottoinsieme di un sistema.

Definizione 3.2: Vista

Una **vista** raffigura determinati aspetti del modello

Definizione 3.3: Notazione

Una **notazione** è un set di regole grafiche o testuali per raffigurare le viste

Esempio Pratico: Aeromobile
Per comprendere meglio la gerarchia tra sistema, modelli e viste, si consideri il caso di un aeromobile:

- **Sistema**: L'aeromobile fisico nella sua interezza.
- **Modelli**: Rappresentazioni come il simulatore di volo o un modello in scala.
- **Viste**: Dettagli specifici del modello, come i progetti tecnici (blueprints), lo schema del cablaggio elettrico o il sistema di alimentazione del carburante.

La validità di un modello è determinata dal mapping tra la realtà (R) e le astrazioni nel modello (M): le relazioni valide nella realtà devono riflettersi correttamente nel modello.

3.2.1 Tipologie di Modelli

- **Modello Funzionale**: Descrive le funzionalità del sistema e il flusso dei dati.
- **Modello Oggetto (Strutturale)**: Descrive la struttura del sistema, gli oggetti e le loro relazioni.
- **Modello Dinamico (Comportamentale)**: Descrive come il sistema reagisce agli eventi esterni.

3.3 UML e le 4+1 Viste

Tra la fine degli anni '80 e l'inizio degli anni '90, sono emerse numerose metodologie orientate agli oggetti, ciascuna con le proprie notazioni e i propri processi di sviluppo. Gli approcci più rilevanti (Booch, Rumbaugh e Jacobson) sono stati unificati nell'UML (Unified Modeling Language).

L'UML è un formalismo di modellazione standardizzato dall'OMG (Object Management Group) che racchiude diversi linguaggi e permette di costruire modelli per l'analisi, il progetto e l'implementazione. Dal punto di vista architetturale, UML definisce 4+1 viste fondamentali:

- **Use Case View** (Vista dei casi d'uso) [cite: 89]
- **Logical/Design View** (Classi, interfacce, collaborazioni)
- **Process View** (Vista dei processi) [cite: 90]
- **Component/Implementation View** (Componenti fisici del software)
- **Deployment View** (Distribuzione del software sull'hardware)

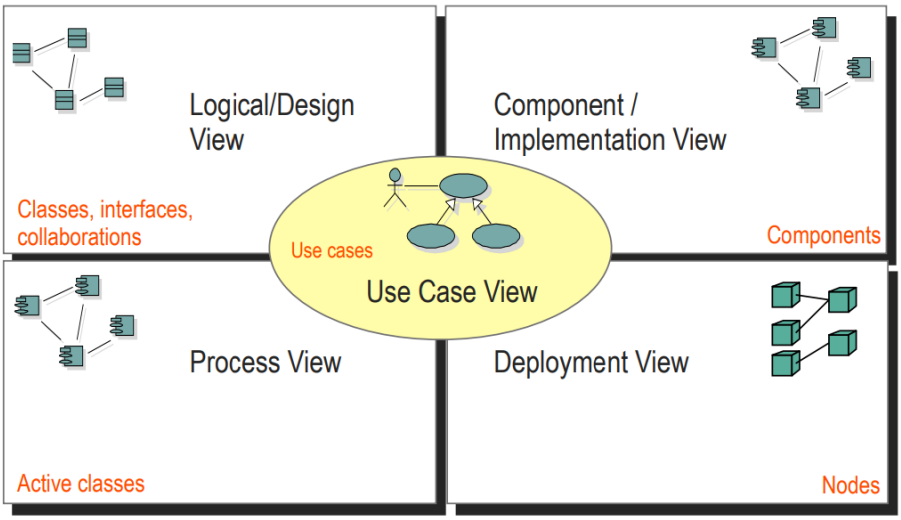


Figura 3.1: 4+1 viste

3.4 Elicitazione dei Requisiti

L'**elicitazione dei requisiti** è un'attività complessa e critica che richiede la stretta collaborazione tra professionisti con competenze eterogenee: **utenti** (detentori della conoscenza del dominio applicativo) e **sviluppatori** (esperti del dominio della soluzione, del design e dell'implementazione).

Per colmare la lacuna comunicativa tra utenti e sviluppatori si introducono due strumenti chiave:

- **Scenari**: Rappresentano esempi concreti di utilizzo del sistema, descritti come una serie di interazioni passo-passo tra l'utente e il sistema stesso.
- **Casi d'uso (Use Cases)**: Costituiscono un'astrazione che raggruppa e descrive un'intera classe di scenari correlati. Rappresentano le funzionalità tangibili offerte all'utente dal sistema.

3.5 Diagramma dei Casi d'Uso - Modellazione funzionale

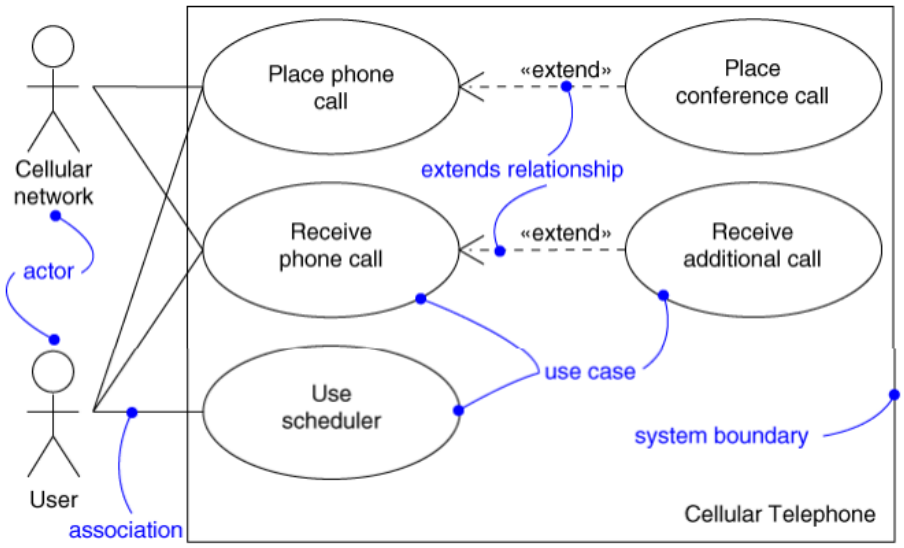


Figura 3.2: Use case diagram: Functional modelling

È il diagramma dedicato alla modellazione funzionale del sistema. Gli elementi principali sono:

- **Attore**: Modella un'entità esterna che comunica direttamente con il sistema. Può essere un utente umano, un sistema esterno o l'ambiente fisico (es. un sensore).
- **Caso d'Uso**: L'azione o la funzione messa a disposizione.
- **System Boundary**: I confini del sistema che racchiudono i casi d'uso, separandoli dagli attori esterni.

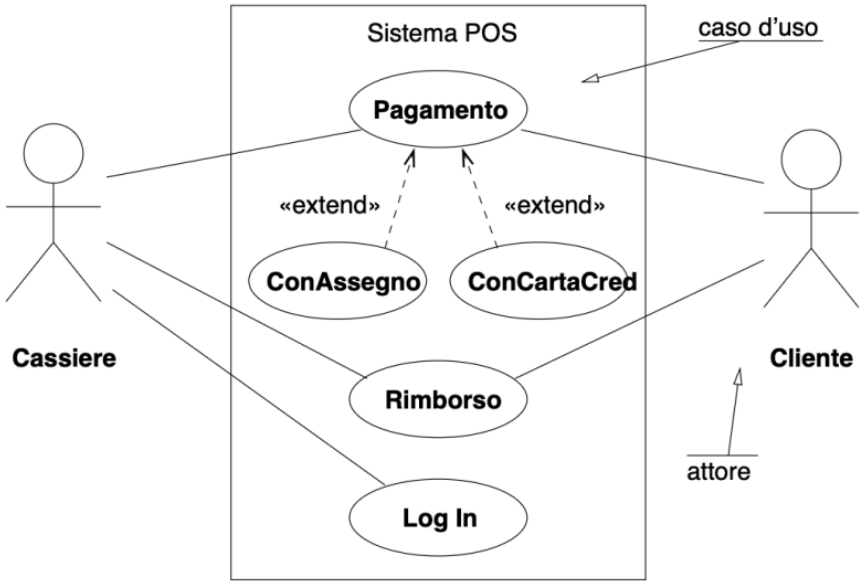


Figura 3.3: Esempio di diagramma di caso d'uso

I casi d'uso possono essere correlati tra loro tramite specifiche relazioni:

- **«include»**: Indica che il comportamento del caso d'uso incluso è obbligatorio e viene richiamato all'interno del caso d'uso base.
- **«extend»**: Specifica che un caso d'uso base può essere opzionalmente esteso con ulteriori comportamenti, generalmente al verificarsi di determinate condizioni.

3.6 Il Progetto Software Orientato agli Oggetti

Il modello derivante dalla fase di analisi funge da base per la definizione dell'architettura software. Nelle fasi inziali, l'architettura ricalca fedelmente il **modello di analisi**. La fase di progettazione vera e propria **estende** le classi trovate in fase di analisi, integrandole con classi derivanti dalla progettazione del sistema e inerenti al dominio dell'implementazione (es. database, interfacce utente). Questa struttura viene poi rielaborata, introducendo nuove classi, espandendo i concetti in moduli complessi e definendo accuratamente le interfacce per preparare il codice.

3.6.1 Obiettivi Architetturali

La progettazione copre due livelli architetturali:

1. **Architettura logica del software**: Include la definizione di moduli, sottosistemi, componenti e moduli unitari. Definisce inoltre le relazioni formali tra essi: associazioni, generalizzazioni, dipendenze e collaborazioni.
2. **Architettura fisica**: Riguarda l'organizzazione dei file eseguibili, delle librerie e dei dati, unitamente all'architettura hardware (nodi computazionali e ambienti di esecuzione).

3.6.2 Tipologie di Diagrammi UML

I diagrammi UML si dividono in due macrocategorie[cite: 217]:

- **Diagrammi Strutturali (Structure Diagram)**: Mostrano l'architettura statica. Includono: Class diagram, Component diagram, Composite structure diagram, Deployment diagram, Object diagram, Package diagram.
- **Diagrammi Comportamentali (Behavior Diagram)**: Mostrano l'evoluzione dinamica. Includono: Activity diagram, Use case diagram, State machine diagram, Interaction diagram (Sequence, Communication, Interaction overview, Timing diagram).

3.7 Diagramma delle Classi e degli Oggetti

3.7.1 Attributi e Operazioni

La sintassi formale per dichiarare un attributo in un diagramma delle classi è:

```
<visibilità> <nome> [<molteplicità>] :  <tipo> = <valore-iniziale>
```

La visibilità definisce il livello di incapsulamento ed è rappresentata dai simboli:

- `+` pubblica (public)
- `#` protetta (protected)
- `~` package (visibilità di default in Java)
- `-` privata (private)

La molteplicità, indicata tra parentesi quadre, definisce **quanti valori** può assumere l'attributo: es. `[3]` (esattamente tre), `[1..4]` (da uno a quattro), `[1..*]` (uno o più), `[0..1]` (attributo opzionale). Se un attributo possiede uno scope statico, il suo nome viene sottolineato nel diagramma.

La sintassi per definire un'*operazione* (**metodo**) è:

```
<visibilità> <nome> (<lista-parametri>) :  <tipo-restituito>
```

Il nome e la lista dei parametri costituiscono la **segnatura** dell'operazione. I parametri si specificano tramite:

```
<direzione> <nome> :  <tipo> = <valore-default>
```

dove la direzione può essere `in` (ingresso), `out` (uscita) o `inout` (ingresso e uscita). Il valore di default viene utilizzato se non viene passato alcun argomento al metodo in fase di invocazione.

3.7.2 Oggetti

Mentre i diagrammi di classe modellano le regole generali, gli oggetti si usano per **esemplificare** situazioni particolari o stati specifici. Nel diagramma degli oggetti, il nome dell'oggetto e la sua classe vengono scritti sottolineati e separati dai due punti (es.

`oggetto:Classe` oppure semplicemente `:Classe` se anonimo) e si assegnano valori concreti ai rispettivi attributi.

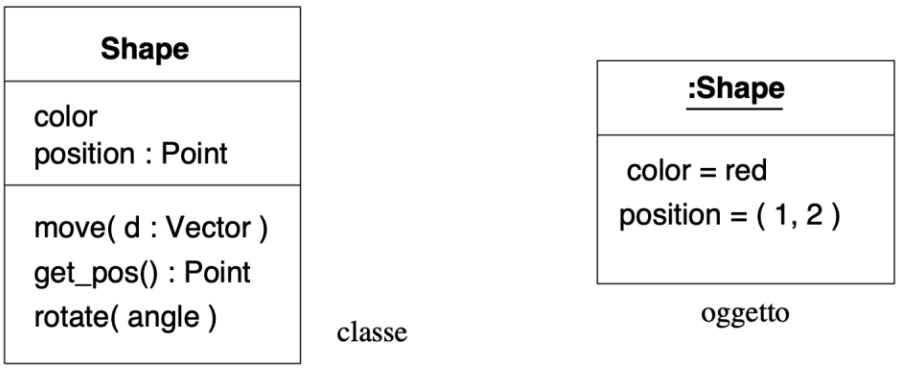


Figura 3.4: Esempio: Oggetti

3.8 Associazioni e Progettazione delle Relazioni

Le associazioni descrivono i collegamenti tra le classi. Un'associazione binaria è definita dai suoi estremi (*end*), ciascuno dei quali specifica:

- Nome della classe
- Ruolo della classe all'interno dell'associazione
- Molteplicità (numero di istanze associabili)
- Navigabilità (presenza di puntatori per attraversare la relazione)

In fase di progettazione, le associazioni si implementano nel codice object-oriented secondo i seguenti pattern:

- **Uno a Uno (1 a 1)**: Si implementa tramite un semplice puntatore (riferimento a oggetto) se la navigazione è monodirezionale, o con due puntatori (uno in ciascuna classe) se bidirezionale.

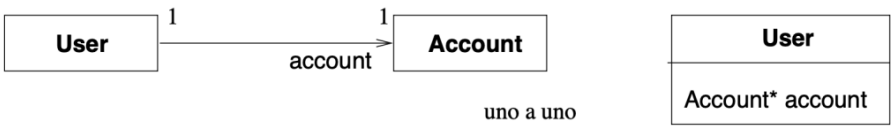


Figura 3.5: Uno a uno

- **Uno a Molti (1 a *)** : Viene implementata utilizzando una classe contenitore. È importante basare la scelta del contenitore sull'eventuale ordinamento richiesto e sulle modalità di accesso ai dati (ad esempio, l'utilizzo di iteratori). In Java, si sfrutta un'interfaccia o una classe del framework *Collections*, inserendo una lista (es. `List<Account>`) all'interno dell'entità principale.

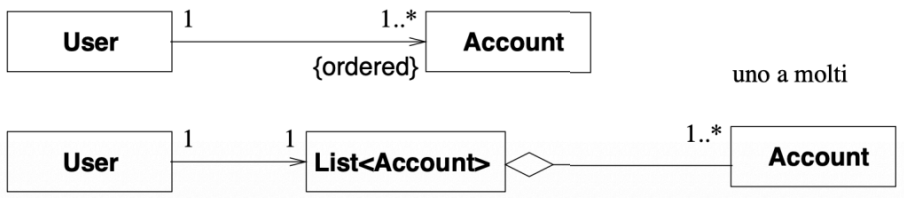


Figura 3.6: Uno a molti

- **Molti a Molti (* a *)**: La relazione deve essere risolta implementando una classe intermedia che scompone l'associazione originaria in due associazioni del tipo "uno a molti". Questa classe conterrà riferimenti alle istanze delle classi collegate.

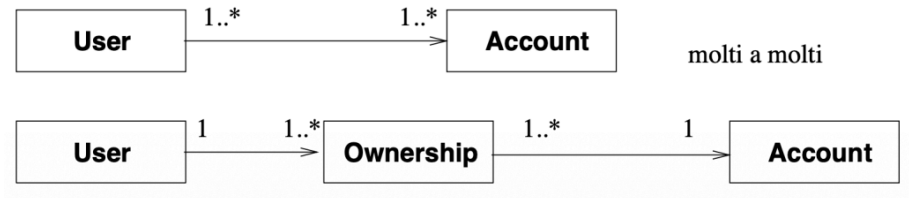


Figura 3.7: Molti a molti

3.8.1 Associazioni Qualificate

Un'associazione qualificata si avvale di un **qualificatore**, ovvero un attributo specifico dell'associazione che viene utilizzato come "*chiave*" per individuare univocamente un singolo oggetto all'interno di una vasta collezione di oggetti collegati. (Es. In un dizionario, la stringa della parola agisce da qualificatore per recuperare il valore).

3.8.2 Relazioni Speciali

- **Dipendenza**: **Si ha quando una classe dipende in modo lasco da un'altra**. Si verifica tipicamente se un'operazione della classe prende l'altra come parametro, ne restituisce un'istanza, o crea dinamicamente un suo oggetto locale che non è salvato come attributo di stato.

- **Aggregazione**: **Esprime il concetto di appartenenza a un gruppo** (relazione *whole/part*). È un'associazione "debole" in quanto le parti componenti possono esistere in modo del tutto indipendente al di fuori dell'entità composta (es. una squadra e i suoi giocatori, una biblioteca e i libri).
  Si disegna con un rombo vuoto adiacente alla classe contenitore. Un'istanza di una classe può partecipare simultaneamente a più di un'aggregazione.

- **Composizione**: È una versione molto più stringente dell'aggregazione. Modella una **subordinazione strutturale rigida**, in cui l'esistenza delle parti è strettamente legata e dipendente dal contenitore (es. il motore di un'automobile).
  Si rappresenta tramite un rombo pieno. Un'istanza componente può partecipare a una e una sola composizione.

- **Generalizzazione (Ereditarietà)**: È una **relazione strutturale** (non un'associazione) che indica che una classe (sottoclasse) è una specializzazione o un sottoinsieme di un'altra (classe base). L'affermazione "*A generalizza B*" o "*B specializza*

A" equivale a dire che ogni istanza di B è di fatto un'istanza di A (inclusione logica di insiemi). Su di essa si poggia il **principio di sostituzione di B. Liskov**, secondo il quale *un'istanza della classe derivata deve poter sempre sostituire un'istanza della classe base garantendo il corretto funzionamento del software.*

Le classi coinvolte nelle generalizzazioni si dividono in classi

- **concrete**:se possono essere **direttamente** istanziate
- **astratte**: se dichiarano comportamenti ma non possono avere istanze dirette; in UML il nome è in corsivo.

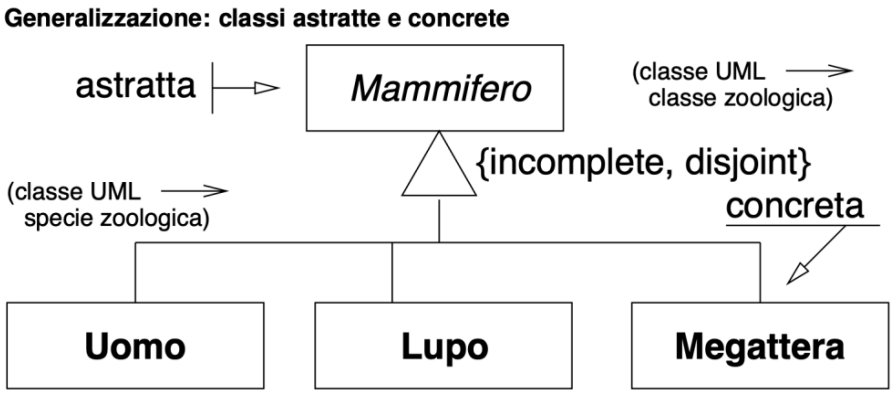


Figura 3.8: Generalizzazione: classi astratte e concrete

3.9 Esempio di Diagram Class

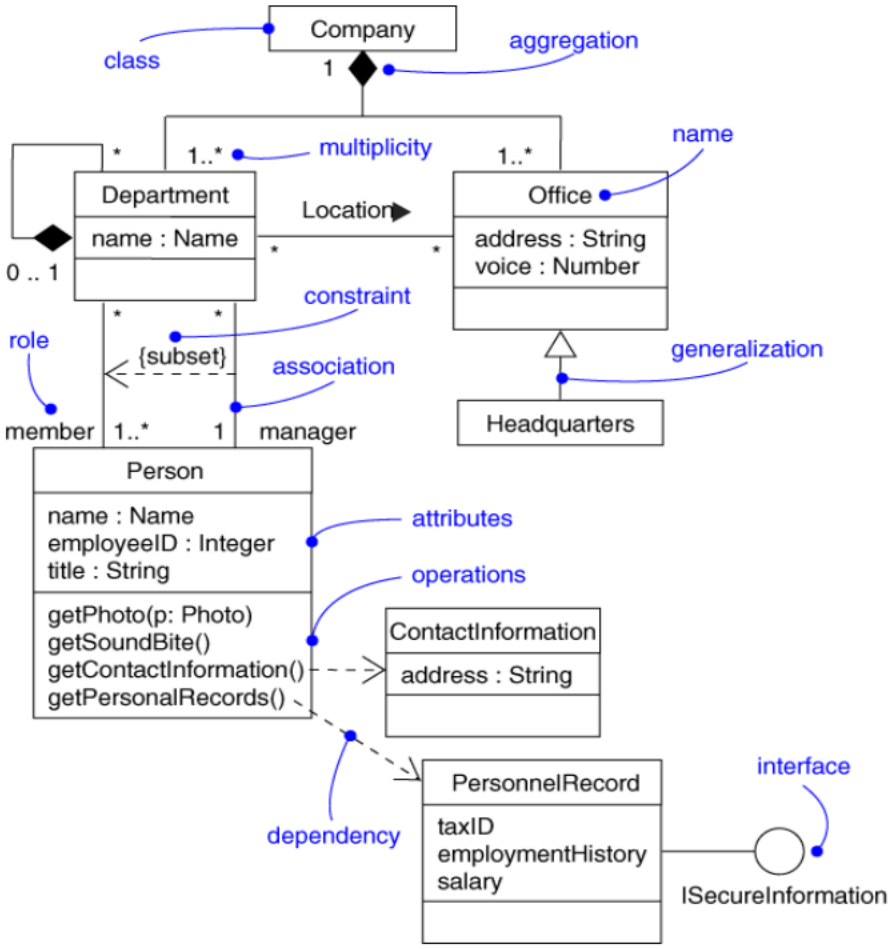


Figura 3.9: Esempio di Diagram Class

Durante la transizione verso il progetto OO di dettaglio, i modelli devono essere affinati per la scrittura del codice. Il software engineer deve:

- Scomporre le classi e i moduli troppo complessi in entità più semplici ed adese (Single Responsibility Principle).
- Specificare completamente i contratti per attributi e operazioni, dettagliando tipi di dato, direzioni dei parametri, visibilità e modificabilità.
- Inserire operazioni essenziali che sono implicite a livello di modello astratto ma necessarie in fase implementativa, come costruttori, distruttori e metodi *getter/setter*.
- Definire eventuali operazioni e interfacce ausiliarie richieste dall'infrastruttura sottostante.

3.10 Modellazione Comportamentale in UML

L'UML offre diversi formalismi per modellare il comportamento dinamico di un sistema orientato agli oggetti. I principali diagrammi comportamentali sono:

- Macchine a Stati (**State Machines**)
- Diagrammi di Attività (**Activities**)
- Diagrammi di Interazione (**Interactions**)

3.11 Diagrammi di Stato (State Machines)

Le macchine a stati descrivono come un oggetto o un sottosistema risponde agli eventi nel corso del suo ciclo di vita.

- Rappresentano un'estensione degli automi a stati finiti.
- Il linguaggio utilizzato in UML deriva direttamente dal formalismo degli **Statecharts** introdotto da David Harel.
- Possono essere associate a una singola classe, a una collaborazione (interazione di un insieme di oggetti) o all'esecuzione di un metodo specifico.

3.11.1 Elementi Fondamentali

- **Stato**: Descrive un intervallo di tempo durante la vita di un oggetto. È caratterizzato dai valori assunti dall'oggetto, dall'attesa del verificarsi di certi eventi o dall'esecuzione di specifiche attività.

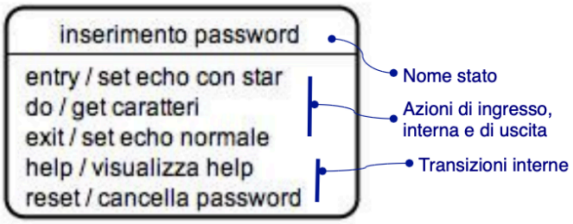


Figura 3.10: Esempio di Stato

- **Occorrenza ed Evento**: Un'occorrenza è associata a un istante nel tempo e (spesso implicitamente) a un punto nello spazio, come la singola pressione di un tasto. Un **evento** definisce l'insieme di occorrenze di uno stesso tipo. Spesso i due termini vengono usati come sinonimi.
- **Transizione**: Modella il passaggio istantaneo da uno stato all'altro in conseguenza al verificarsi di un evento. Permette di lasciare uno stato in risposta a un trigger. **Nota bene:** Un oggetto gestisce un solo evento alla volta.
- **Attività**: Possono essere associate agli stati o alle transizioni. Le attività associate agli stati hanno una durata non nulla e possono essere interrotte da eventi che causano l'uscita dallo stato. Le attività associate alle transizioni (**azioni**) non sono interrompibili e sono considerate istantanee.

3.11.2 Sintassi delle Transizioni e Tipi di Eventi

Le transizioni sono annotate con la seguente sintassi, dove tutti gli elementi sono facoltativi:

```
<evento> [<condizione_di_guardia>] / <azione>
```

- **Trigger (Evento di inizio)**: L'evento che innesca la transizione. Può essere la ricezione di chiamate di operazione o segnali.
- **Condizione di guardia (Guard)**: Espressione booleana racchiusa tra parentesi quadre. È valutata al momento dell'evento; la transizione avviene **solo** se è valutata a `true`.
- **Azione (Action)**: Attività eseguita istantaneamente durante la transizione verso lo stato target.

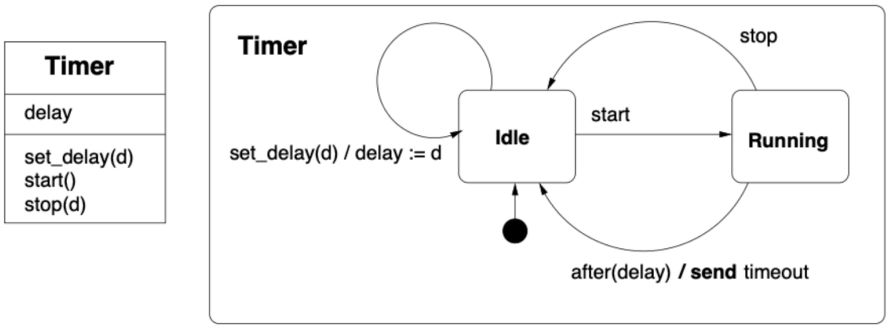


Figura 3.11: Diagramma di Stato

Oltre agli eventi di ricezione, UML supporta:

- **Eventi di cambiamento (Change Events)**: Scattano quando una condizione logica diventa vera, indicati con `when(...)` (es. `when(t > t_max)`).
- **Eventi temporali**: Possono essere assoluti (es. `at(time=2026-3-1 00:00)`) o relativi all'istante in cui lo stato è diventato attivo (es. `after(20 s)`).
- **Eventi di completamento**: Generati automaticamente al termine dell'attività interna di uno stato.

3.11.3 Azioni Interne agli Stati

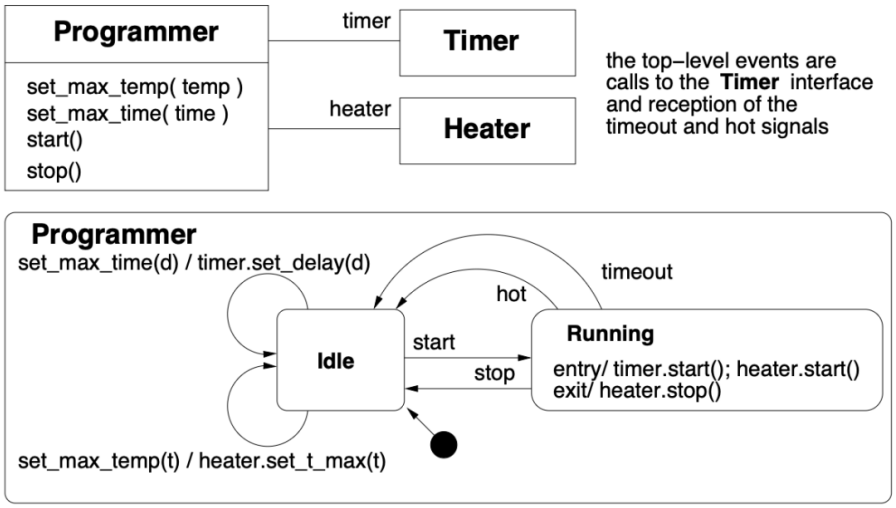


Figura 3.12: Azioni di *entry* e *exit*

All'interno di uno stato, possono essere definite azioni specifiche:

- `entry/`: Azioni eseguite immediatamente all'ingresso nello stato.
- `exit/`: Azioni eseguite immediatamente prima di uscire dallo stato.
- `do/`: Attività in esecuzione mentre ci si trova in un determinato stato (può essere un ciclo continuo, es. `do/ get caratteri`).

3.11.4 Macchine a Stati Gerarchiche e Concorrenti

In UML, le macchine a stati possono essere modellate gerarchicamente per gestire la complessità. Il comportamento di un **superstato** può essere dettagliato attraverso un insieme di sottostati.

Figura 3.13: Stati composti

- **Sottostati Sequenziali**: All'interno del superstato, vi è un solo stato attivo alla volta.
- **Ereditarietà delle transizioni**: I sottostati ereditano le transizioni in uscita definite a livello del superstato. Ad esempio, in un sistema telefonico, se il superstato "Attivo" ha una transizione verso "Inattivo" scatenata dall'evento "sconnessione", tutti i sottostati (Attesa, Conversazione, ecc.) risponderanno a quell'evento passando a "Inattivo".

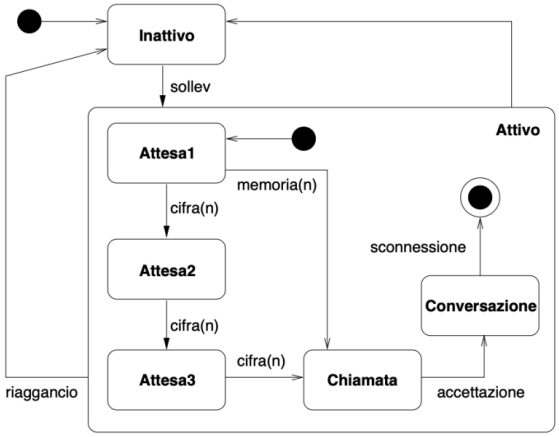


Figura 3.14: Macchina a Stati gerarchica

- **Regioni Concorrenti**: I sottostati possono essere concorrenti, ovvero due o più stati possono essere attivi contemporaneamente in partizioni ortogonali del sistema (es. in un dispositivo "Operation", i sottosistemi per la velocità e la temperatura evolvono in modo parallelo).

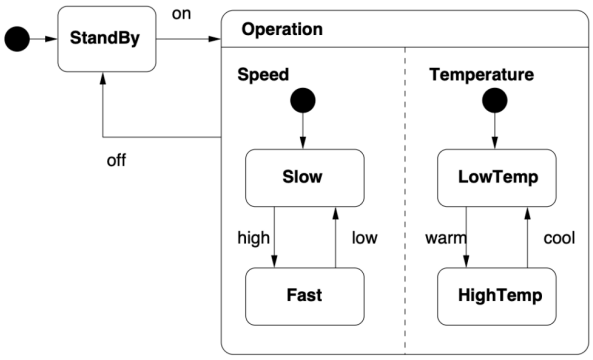


Figura 3.15: Macchina a Regioni concorrenti

Le transizioni concorrenti utilizzano **fork** (diramazione) e **join** (sincronizzazione) per gestire l'attivazione e la terminazione dei thread paralleli.

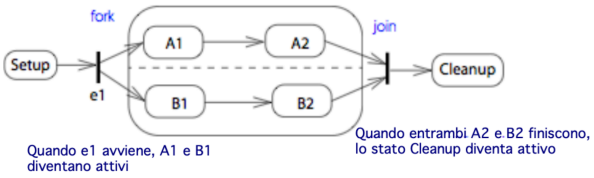


Figura 3.16: Enter Caption

3.12 Diagrammi di Attività

I diagrammi di attività servono a descrivere il flusso di controllo e di informazioni all'interno dei processi. Nella loro forma più semplice, sono concettualmente molto simili ai tradizionali diagrammi di flusso (flowchart).

- **Nel modello di analisi**: Descrivono i processi del dominio applicativo, come la gestione aziendale o l'elaborazione di ordini e pagamenti.
- **Nel modello di progetto**: Descrivono flussi algoritmici complessi o l'implementazione logica delle singole operazioni.
- Costituiscono il punto iniziale della progettazione. Tuttavia, i diagrammi di base non esplicitano l'oggetto responsabile di una data attività; devono quindi essere ri-elaborati per l'assegnazione delle responsabilità alle classi.

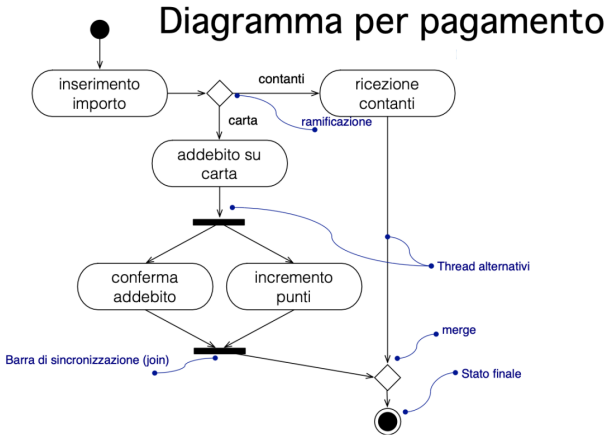


Figura 3.17: Diagramma per pagamento

3.12.1 Struttura e Corsie (Swimlanes)

Per modellare l'assegnazione delle responsabilità nel processo, si utilizzano le **Corsie (Swimlanes)**.

- **Corsie**: Partizioni verticali o orizzontali che indicano l'attore, l'oggetto o il modulo responsabile di uno specifico sottoinsieme di attività (es. Cliente, Vendite, Magazzino).
- **Flusso di Oggetti**: I dati o gli oggetti di dominio utilizzati o prodotti dalle attività sono rappresentati come rettangoli. Le frecce tratteggiate indicano l'input o l'output di tali oggetti verso o da un'attività.
- **Ramificazione e Sincronizzazione**: Vengono utilizzati **nodi di decisione** per i thread alternativi e **barre di sincronizzazione (fork/join)** per la gestione dell'esecuzione parallela. Il nodo di **merge** ricongiunge percorsi mutuamente esclusivi.

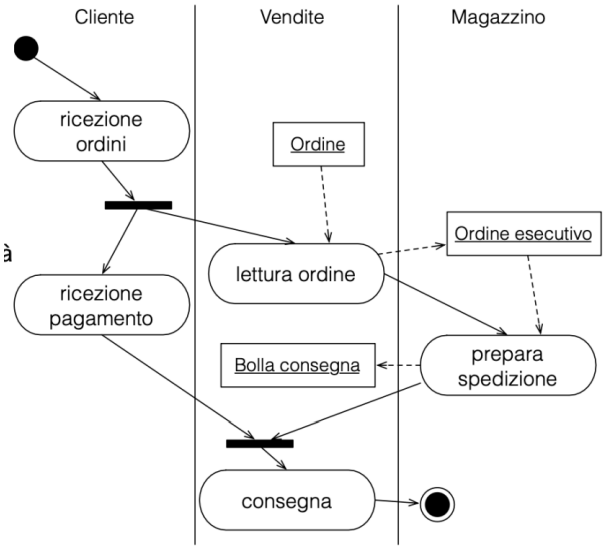


Figura 3.18: Diagramma attività con Corsie

3.13 Diagrammi di Interazione

L'UML descrive come gli oggetti collaborano scambiandosi messaggi attraverso i Diagrammi di Interazione, che comprendono i **Diagrammi di Collaborazione** (Comunicazione) e i **Diagrammi di Sequenza**.

3.13.1 Diagrammi di Collaborazione (Comunicazione)

Questi diagrammi mettono in evidenza la **struttura spaziale e topologica** di un'interazione.

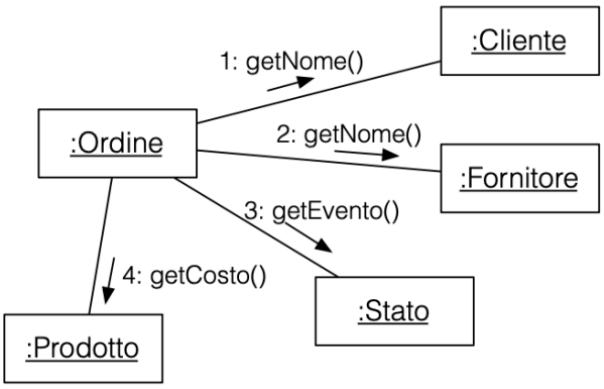


Figura 3.19: Diagramma di collaborazione

- Mostrano esplicitamente i legami e le associazioni fra le istanze che partecipano allo scenario.
- Il flusso dei messaggi è rappresentato da frecce poste parallelamente alle linee di associazione tra gli oggetti.

- L'ordinamento temporale è indicato in modo esplicito attraverso un sistema di **numerazione sequenziale** sulle etichette dei messaggi (es. `1: getNome()`, `2: getName()`).
- Le etichette dei messaggi possono includere il metodo chiamato ed eventuali valori di ritorno.

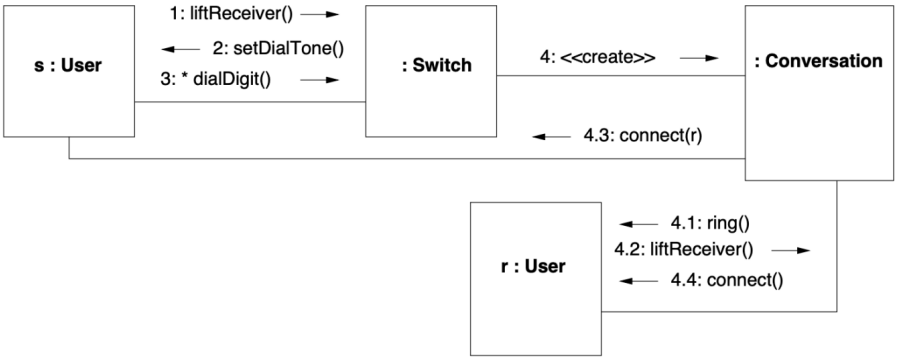


Figura 3.20: Esempio di diagramma di Collaborazione

3.13.2 Diagrammi di Sequenza

A differenza dei diagrammi di collaborazione, i diagrammi di sequenza privilegiano la visualizzazione della **successione temporale** dei messaggi scambiati tra le "linee di vita" degli oggetti nel tempo (asse verticale).

- Rappresentano scenari specifici e limitati, ovvero possibili istanze singole del comportamento di un sistema interagente.
- A differenza dei diagrammi di stato (che descrivono **tutte** le possibili evoluzioni di un **singolo** oggetto), un diagramma di sequenza fissa uno specifico scenario (es. lo scenario in cui viene raggiunto il timeout prima del limite di temperatura).

Mappatura codice Java / Diagrammi UML

Il formalismo UML mappa in modo diretto le operazioni della programmazione a oggetti (es. Java):

1. **Creazione di un'istanza**: L'istruzione Java `Studente s = new Studente();` corrisponde a un messaggio con etichetta `create` indirizzato all'intestazione della nuova istanza `s:Studente`.

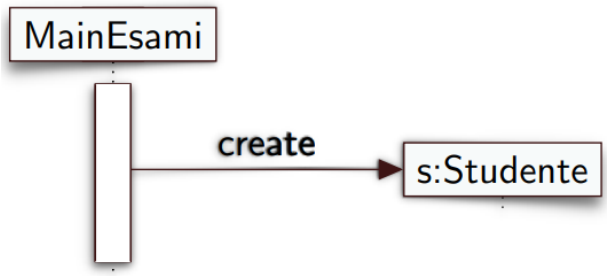


Figura 3.21: Creazione di un'istanza

2. **Chiamata di metodo**: L'istruzione Java `s.setNome("Alan", "Rossi");` corrisponde a una freccia orientata verso la linea di vita dell'oggetto `s:Studente`, con etichetta `setNome()`.

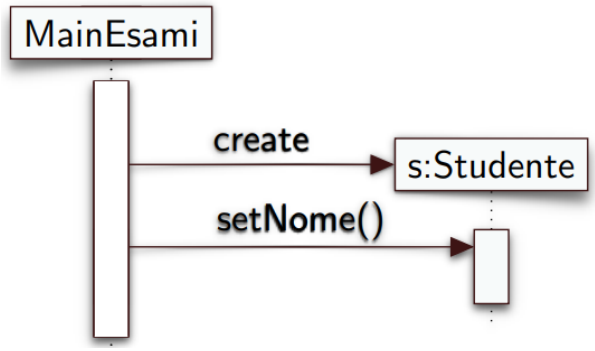


Figura 3.22: Chiamata di metodo su un'istanza

3. **Chiamate a cascata**: Se l'esecuzione di un metodo ne evoca un altro internamente (es. per istanziare un altro oggetto collegato, come un `Esame`), si modelleranno chiamate ricorsive o a catena, rappresentando le frecce in sequenza temporale discendente.

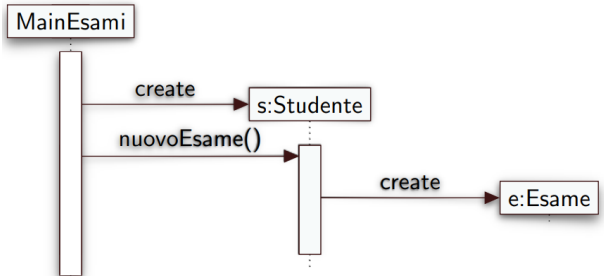


Figura 3.23: Chiamata di metodo a cascata

È bene osservare che ciascun diagramma di sequenza rappresenta esplicitamente una o piú istanze delle possibili sequenze di messaggi, mentre un diagramma di stato definisce implicitamente tutte le possibili sequenze di messaggi ricevuti (eventi) o inviati (azioni send) da un oggetto interagente con altri.

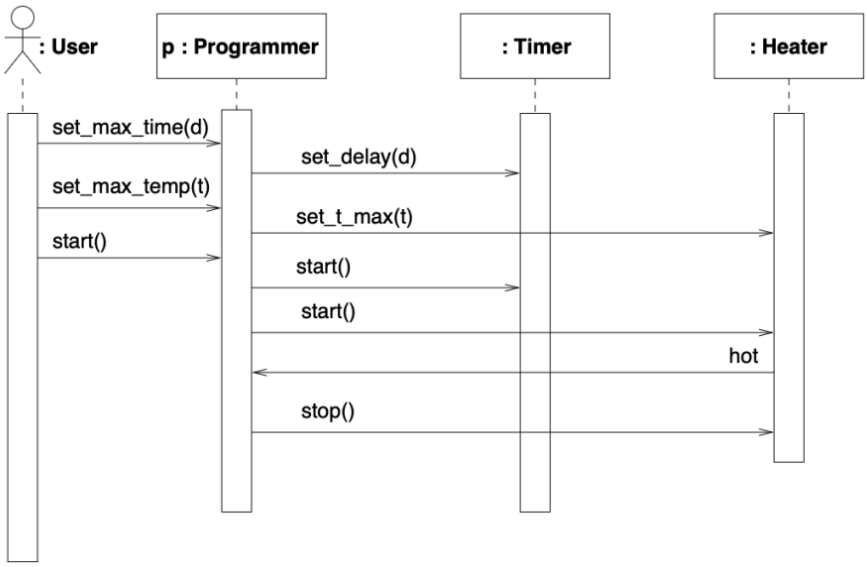


Figura 3.24: Scenario 1: l'heater raggiunge la temperatura limite prima del timeout o dell'interruzione manuale

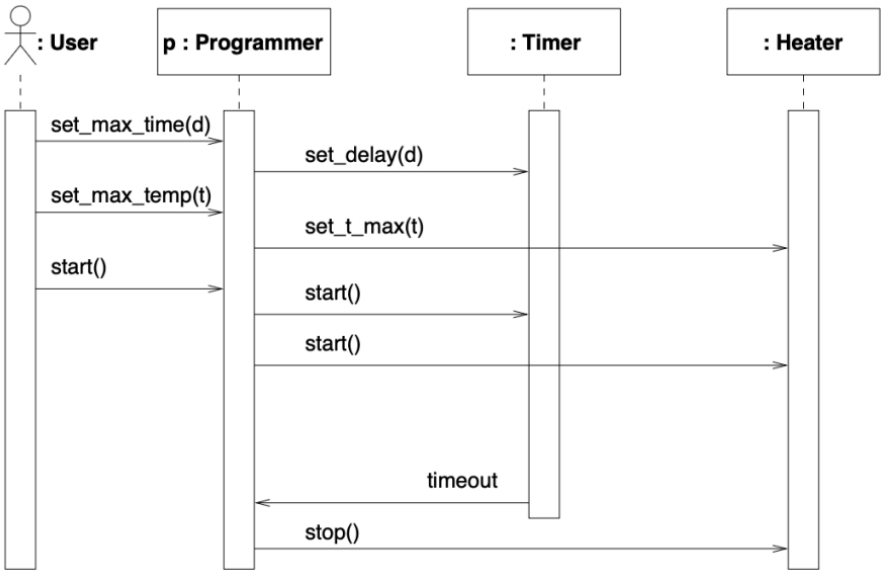
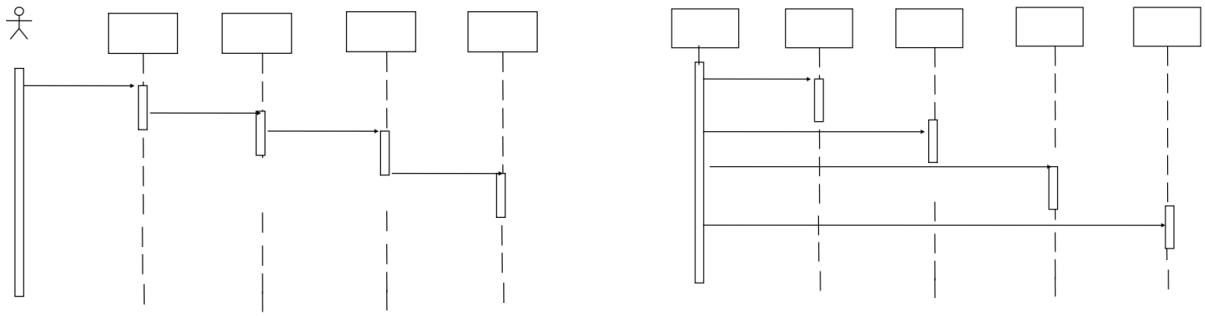


Figura 3.25: Scenario 2: il timeout avviene prima che l'heater raggiunga la temperatura limite



(a) Stair Diagram (b) Fork Diagram

Figura 3.26: Tipologie di Diagrammi

Frammenti e Flussi di Controllo

Per rappresentare logica di controllo complessa all'interno dei diagrammi di sequenza, UML utilizza il concetto di **frammento d'interazione**. Ogni frammento è delimitato da un rettangolo e contrassegnato da una parola chiave in alto a sinistra:

- `opt` (Optional): Segnala una sequenza eseguita condizionalmente, solo se una definita clausola di guardia è soddisfatta.
- `loop` (Ciclo): Definisce una sequenza ripetuta un numero specificato o indeterminato di volte, finché sussiste una condizione logica specificata nell'intestazione.
- `par` (Parallel): Delimita regioni distinte contenenti sequenze di messaggi che vengono eseguite interamente in parallelo tra loro.
- `critical`: Delimita una porzione di sequenza atomica e non interrompibile, garantendo la mutua esclusione.

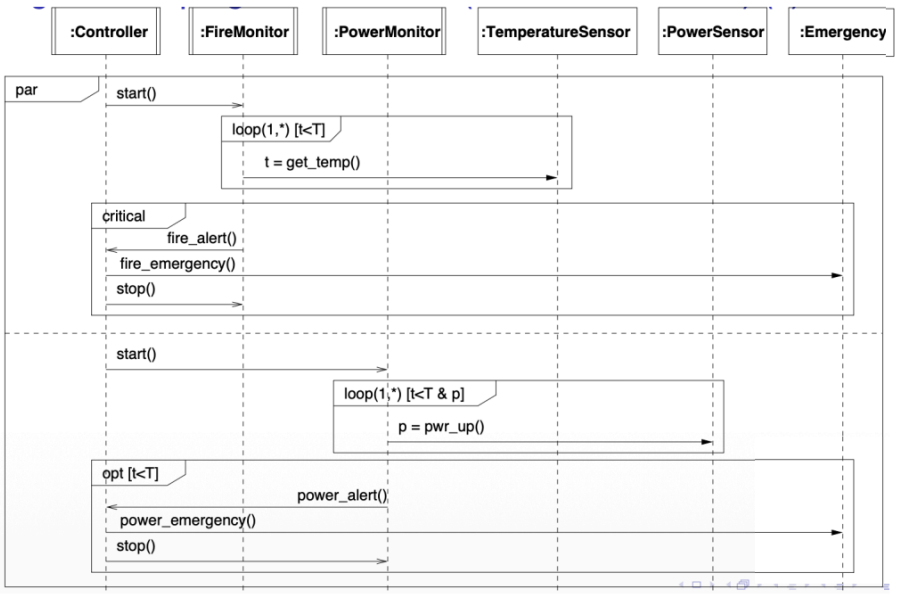


Figura 3.27: Esempio di flussi di controllo nei diagrammi di sequenza